

Lecture 4: Data Representation III – Struct Layout & Alignment

Session 4 · Wed Sep 9, 2026

Warm-Up: Printing a Number in Binary

C has no `%b` conversion. If you want to see the bits, you write it yourself — and everything you need is from Lecture 2:

```
void print_binary(uint32_t x)
{
    for (int i = 31; i >= 0; i--) {
        putchar((x & (1u << i)) ? '1' : '0');
        if (i % 4 == 0 && i != 0)
            putchar(' ');           // one space per nibble = one hex digit
    }
}
```

- `1u << i` builds a **mask** — one bit set, at position `i`. `1u`, not `1`: `1 << 31` is undefined on a 32-bit `int`.
- `x & mask` **tests** that bit: nonzero if set, zero if not.
- Counting down from 31 prints the high bit first, the way we write numbers.

Run It: Old Friends in Binary

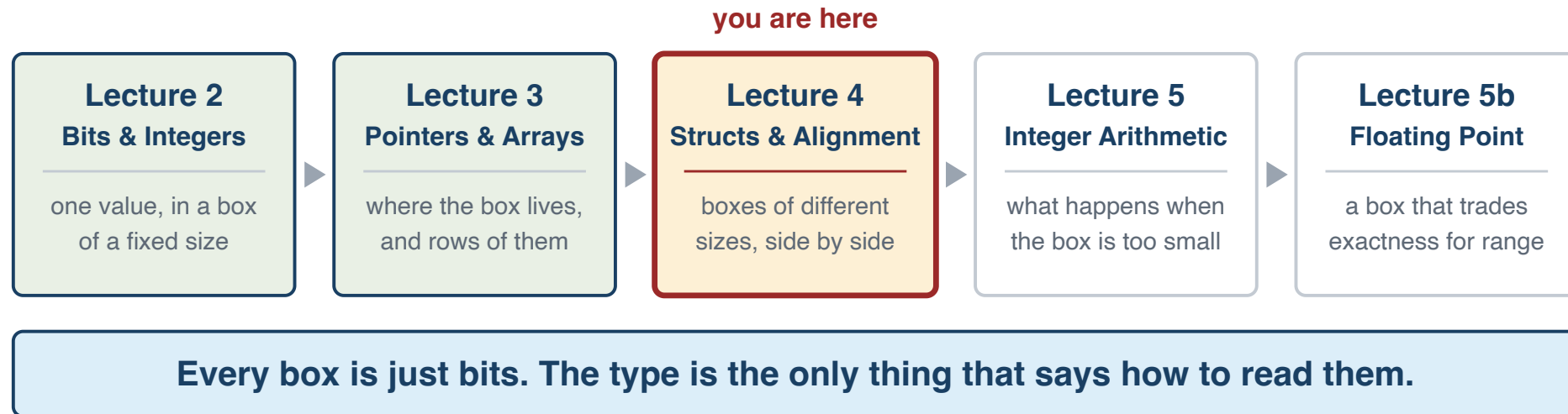
5	00000005	0000	0000	0000	0000	0000	0000	0000	0101
42 (0x2A)	0000002A	0000	0000	0000	0000	0000	0000	0010	1010
1u << 31	80000000	1000	0000	0000	0000	0000	0000	0000	0000
0xFF00FF00	FF00FF00	1111	1111	0000	0000	1111	1111	0000	0000
-1	FFFFFFFF	1111	1111	1111	1111	1111	1111	1111	1111
-5	FFFFFFFB	1111	1111	1111	1111	1111	1111	1111	1011
INT32_MIN	80000000	1000	0000	0000	0000	0000	0000	0000	0000
(char)-5 -> int	FFFFFFFB	1111	1111	1111	1111	1111	1111	1111	1011

Read **FF00FF00** against the groups of four — each hex digit is one group. **1u << 31** and **INT32_MIN** are the same pattern, and so are the last two lines: that is sign extension, widening a **char** without changing what it means.

Keep this function. You will want it from here on.

Where We Are

Five lectures, one idea: a value is bits in a box



Lecture 2 filled in one box; Lecture 3 gave it an address. Today: boxes of **different sizes**, side by side — and the gaps the machine leaves between them.

Three Arrays, or One?

A student has a name, an ID and a GPA — so far, three arrays you keep in step by hand. Sort one, forget the others, and the roster is silently wrong:

```
char names[100][16];   int ids[100];   float gpas[100];
```

```
struct Student {  
    char  name[16];  
    int   id;  
    float gpa;  
};
```

```
struct Student roster[100];           // one array now, not three  
roster[7].id = 1001;                  // reach a field with a dot
```

`struct Student` is a **type**, like `int`.

typedef: Dropping the Word struct

Writing `struct Student` everywhere gets old. `typedef` gives the type a second name:

```
typedef struct {  
    char  name[16];  
    int   id;  
    float gpa;  
} Student;  
  
Student roster[100];           // no "struct" needed
```

Most C code you read does this. It changes nothing about the layout — the same padding rules apply, byte for byte.

Filling One In

Three ways, and the third is the one to prefer:

```
struct Student b = { "Beto", 1002, 3.1f };           // in declaration order  
struct Student c = { .name = "Carmen", .id = 1003 }; // by name
```

The second is a **designated initializer**: order does not matter, and anything you leave out is zero. Reorder the struct later and it still means the same thing. Prefer it.

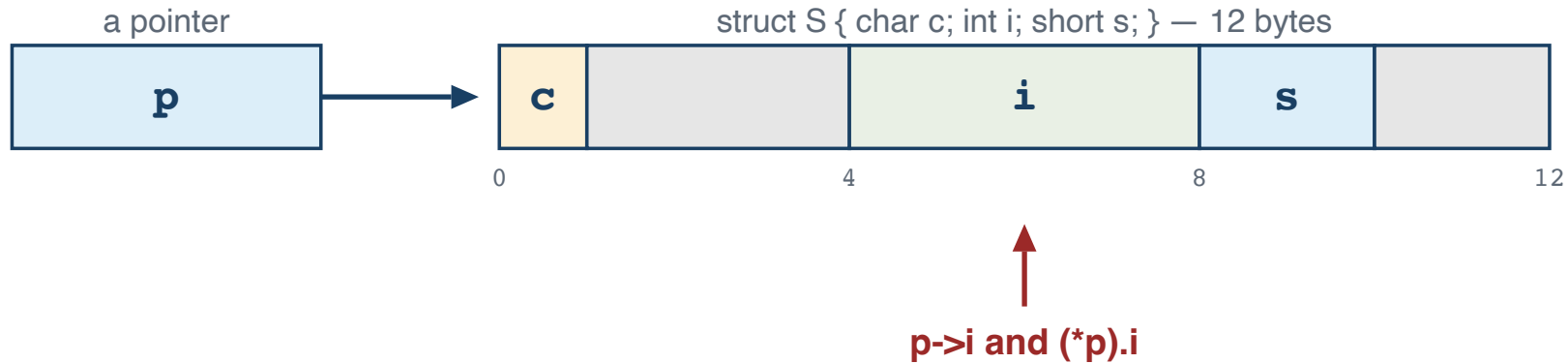


Tip

Assigning one struct to another copies the whole thing — the `char` array inside it included. This is the one place C copies an array for you.

Structs Through a Pointer

p->i is (*p).i, spelled without the parentheses



“.” binds tighter than “*”, so `*p.i` would try to dereference `p.i`. That is why `->` exists.

```
struct Point *p = &pt;
```

```
p->x      // the field x of the struct p points at  
(*p).x   // the same thing, written out
```

Passing One to a Function

```
void move_by_value(struct Point p, double dx) { p.x += dx; } // a copy
void move_by_pointer(struct Point *p, double dx) { p->x += dx; } // the original
```

```
sizeof(struct Point) = 24    sizeof(pointer) = 8
```

```
after move_by_value    (1.0, 2.0) tag 7
after move_by_pointer  (101.0, 2.0) tag 7
```

Passing by value copies all 24 bytes and throws the result away. Pass a pointer, and write `const struct Point *` when the function only reads.

Structs in Memory

Now the part that surprises people. Add up the fields:

```
struct S {  
    char c;    // 1 byte  
    int i;     // 4 bytes  
    short s;   // 2 bytes  
};
```

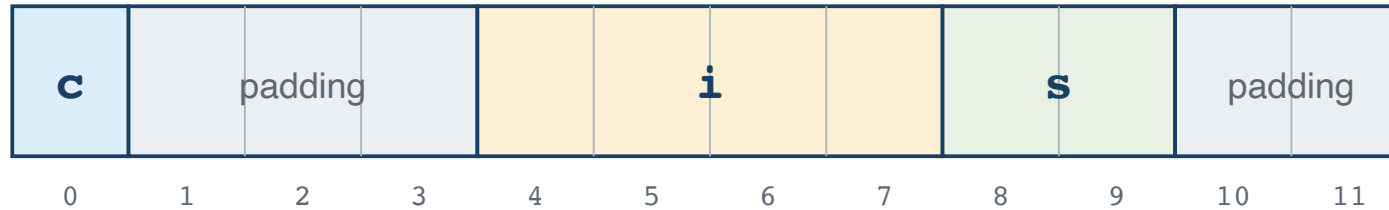
That is 7. Now ask the compiler:

```
printf("%zu\n", sizeof(struct S)); // 12 on our x86-64 teaching systems
```

Five bytes went somewhere. The rest of this lecture is about where, and why.

Where the Five Bytes Went

Three fields that need 7 bytes take 12



The grey bytes are **padding**: nothing is stored in them.

They are there so each field starts at an offset it is allowed to start at.

Two rules produced that picture:

1. Each field starts at a multiple of its **alignment**.
2. The total rounds up to a multiple of the largest alignment in the struct, so that in an array every element stays aligned too.

Alignment Requirements

On our x86-64 teaching systems:

Type	Size	Must start at a multiple of
char	1	1 (anywhere)
short	2	2
int	4	4
double	8	8
pointer	8	8

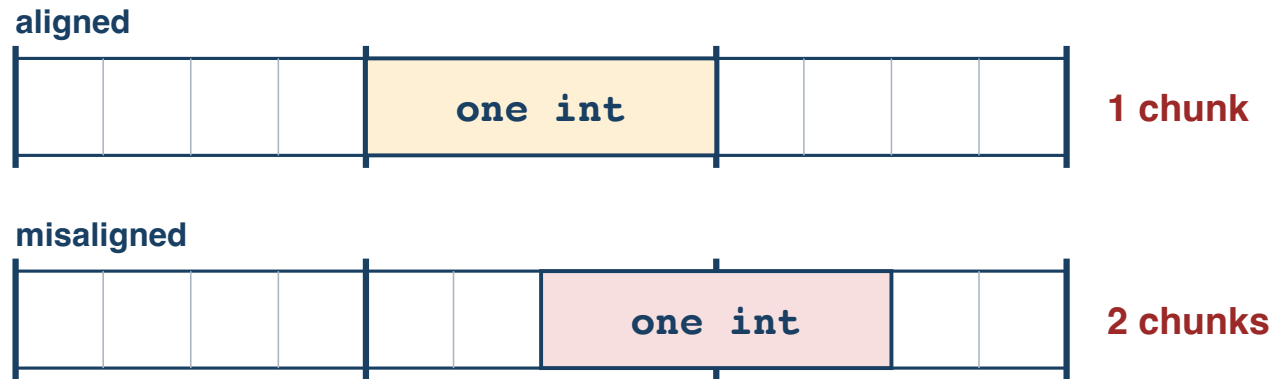
Alignment is not size: `char buf[8]` has size 8 and alignment 1. Ask `_Alignof(type)` when you need the exact figure.

Warning

These numbers are this platform's ABI, not guarantees in the C standard. Print them on the machine you are actually targeting.

Why the Machine Cares

The machine fetches memory in fixed-size chunks



The thick lines are chunk boundaries. Our x86-64 machines will read the bottom one anyway, but it is more work — and the ABI lays structs out so it does not come up.

Memory arrives in fixed-size chunks. An aligned `int` is one chunk; a misaligned one is two, plus the work of gluing the halves. Some architectures do not offer the choice — they fault.

Walking the Rule by Hand

Take the fields in order and apply the two rules. For `struct Bad { char c; int i; short s; }`:

Field	Alignment	Starts at	Occupies
c	1	0	byte 0
i	4	4 — bytes 1, 2, 3 are skipped	bytes 4–7
s	2	8	bytes 8–9

Ten bytes so far. The largest alignment in the struct is 4, so the total rounds up to **12**.

Now do `struct Good { int i; short s; char c; }` yourself: the fields land at 0, 4 and 6, which is seven bytes, rounded up to 8.

Reordering Saves Space

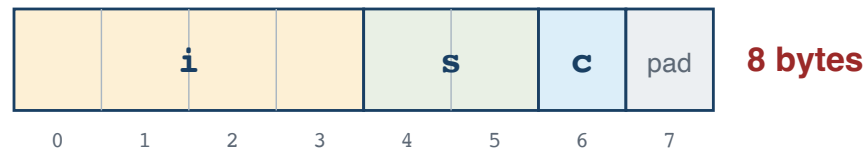
```
struct Bad { char c; int i; short s; }; // 12 bytes here
struct Good { int i; short s; char c; }; // 8 bytes here
```

Same three fields, different order

struct Bad



struct Good



Declaring the widest field first leaves less to pad around. Always confirm with `sizeof`.

Same three fields, same values. Declaring the widest first is the rule of thumb, and it is right most of the time.

The `offsetof` Macro

You do not have to trust your arithmetic. `offsetof` asks the compiler where a field actually landed:

```
#include <stddef.h>
offsetof(struct S, c);    // 0
```

```
struct Bad    sizeof 12 align 4
  c at 0
  i at 4
  s at 8
```

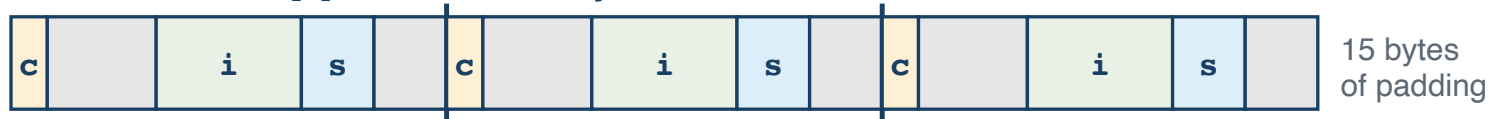
```
struct Good   sizeof 8 align 4
  i at 0
  s at 4
  c at 6
```

The same three numbers you worked out by hand. Four bytes saved per record by typing the same fields in a different order — and when a struct has to match a file format, this is how you check it.

An Array Multiplies It

An array multiplies the padding

`struct Bad roster[3]` — $3 \times 12 = 36$ bytes



`struct Good roster[3]` — $3 \times 8 = 24$ bytes



15 padding bytes become 3 — and over a million records, 4 MB you never load.

1000000 records of Bad	12000000 bytes
1000000 records of Good	8000000 bytes
difference	4000000 bytes

This is why the *trailing* padding exists: without it element 1 would start at offset 10, and its `int` would be misaligned.

Structs Inside Structs

A struct inside a struct is not flattened: the inner one keeps its alignment and hands it outward. And a struct cannot contain *itself* — that would need infinite space — so it holds a pointer instead, 8 bytes whatever it points at:

```
struct Inner { char c; int i; };           // alignment 4
struct Outer { char tag; struct Inner in; char end; };
struct Node  { int value; struct Node *next; }; // alignment 8
```

```
Inner  sizeof 8  align 4  c at 0, i at 4
Outer  sizeof 16 align 4  tag at 0, in at 4, end at 12
Node   sizeof 16 align 8  value at 0, next at 8
```

`in` starts at 4, not 1: a struct's alignment is the largest one anywhere inside it, however deep.
`struct Node` is the shape of every linked list and tree you will write.

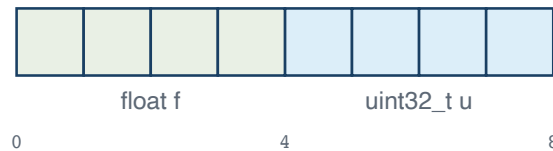
A Union Holds One Member at a Time

A struct is *all of these*. A union is *one of these*:

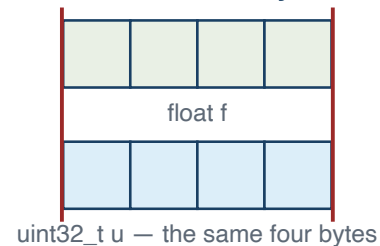
```
struct Both { float f; uint32_t u; }; // both, side by side – 8 bytes
union Bits { float f; uint32_t u; }; // one or the other – 4 bytes
```

Side by side, or on top of each other

struct Both — 8 bytes



union Bits — 4 bytes



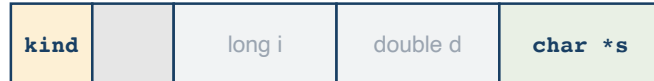
Every union member starts at offset 0. Write one, read another, and you see the same bytes a second way.

Every member starts at offset 0, and the size is settled at **compile time** — the largest member, rounded up to its alignment. It never shrinks to fit: a union of a `char` and a `double` is 8 bytes while it holds the `char`. What is *not* fixed, and not recorded anywhere, is which member is live.

Why You Would Want That

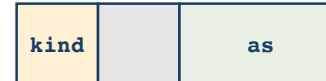
One value that is a long, *or* a double, *or* a string

struct Everything — 32 bytes



kind says TEXT, so i and d are dead weight — and you paid for them anyway.

struct Value — 16 bytes



one 8-byte slot,
three names for it



Half the size — and the tag is the only thing that remembers which name is live.

```
struct Value {  
    enum { V_INT, V_REAL, V_TEXT } kind;    // which member is live  
    union { long i; double d; char *s; } as;  
};
```

The union saves the space; the **tag** remembers what is in it. That pairing is a *tagged union* — how C says “one of these”. Interpreters, JSON parsers and protocol decoders are all built on it.

Read the Member the Tag Names

```
switch (v.kind) {  
case V_INT:  printf("%ld\n", v.as.i);  break;  
case V_REAL: printf("%g\n",  v.as.d);  break;  
case V_TEXT: printf("%s\n",  v.as.s);  break;  
}
```

Read the wrong one and C says nothing at all:

```
wrote u = 42, read f -> 5.88545e-44    (nobody stopped us)
```

That is 42 as integer bits, read as a `float`. The tag is not decoration — it is the only thing standing between you and that number.

Reach for a union when the alternatives are mutually exclusive and you have many of them.

`union sigval` in `<signal.h>` is one: a signal carries either an `int` or a pointer, never both.

The Other Use: The Same Bytes, Read Twice

Sometimes reading the other member is exactly what you want:

```
union Bits { float f; uint32_t u; uint8_t b[4]; };  
  
union Bits v;  
v.f = 3.0f;
```

```
v.f = 3    v.u = 0x40400000  
bytes, lowest address first: 00 00 40 40
```

Well defined in C, and how Lecture 2's endianness test is usually written.



Tip

All of them at once? **struct**. One at a time? **union** plus a tag. The same bytes two ways on purpose? **union** — or `memcpy`, which also works in C++.

When the Layout Is Not Yours to Choose

So far padding has cost you memory. Here is where it costs you the answer.

Whenever a struct must match a layout **someone else defined** — a file header, a network packet, a device register — their rules and the compiler's disagree, and the compiler wins. A BMP header is the smallest real example.

```
the file's header is 14 bytes
struct BmpNaive is 16 bytes    size at offset 4    <- the file says 2

read into BmpNaive:  size = 0    (wrong)
read byte by byte:  size = 58    offset = 54
```

`packed` fixes the size and hands you unaligned fields instead — slower, a fault on some machines, still silent about byte order. Bit fields are no better (`08-bitfields.c`). Assemble the numbers yourself:

```
uint32_t size = (uint32_t)p[0]      | (uint32_t)p[1] << 8
                | (uint32_t)p[2] << 16 | (uint32_t)p[3] << 24;
```

AI Systems Connection

Summing one field of every record

array of structs

struct Particle p[N]; p[i].x is 32 bytes from p[i+1].x



struct of arrays

double xs[N], ys[N], zs[N]; the x values are neighbours



Memory arrives in blocks. On the top row, most of every block is thrown away.

summing x over 8000000 particles, 32 bytes each

array of structs	13.0 ms	256 MB pulled in
struct of arrays	9.3 ms	64 MB pulled in

Array of structs keeps a record together; **struct of arrays** keeps a field together. Every tensor library and every columnar database picks the second — they all read one field of every record. Four times the bytes, only 1.4× the time: the prefetcher hides part of a regular stride. *Lecture 16.*

Your Turn

```
struct Wasteful {  
    char    a;  
    double  b;  
    char    c;  
    int     d;  
    short   e;  
};
```

1. What is `sizeof(struct Wasteful)`? Work it out field by field.
2. Reorder the fields to make it as small as you can. What is the new size?
3. Check both with `sizeof` and `offsetof` before you look at anyone else's answer.

Practice

1. Draw the byte layout of `struct { short a; char b; };` and predict its `sizeof`. Then compile and check.
2. Why does `struct { char c; }` have size 1, but `struct { char c; int i; }` have size 8 rather than 5?
3. Write a function that takes `struct Student *` and returns the GPA. Then write one that takes `struct Student` by value. Which one can change the caller's copy, and why?
4. `08-bitfields.c` declares an IPv4 header with bit fields. Build it. Why is `sizeof` 4 rather than 16, and why does the 16-bit `length` field come out backwards from what the wire format specifies?
5. A `struct Value` holds a tag and a union of `long`, `double` and `char *`. What is its `sizeof`, and where does the union start? Predict, then check.
6. Use a `union` to print the four bytes of `1.0f` in memory order. Which end does the exponent come out at, and why?

Reading

- CS:APP §3.9 (Heterogeneous Data Structures)